Introducing SharePoint Framework

A client-side development model that runs JavaScript in the page, talks to Microsoft 365, and ships the same code to Teams and Viva.

What SPFx is, in one sentence

SharePoint Framework is the page-and-web-part model for client-side SharePoint development. Components run as JavaScript in the current user's browser session, render in the live page DOM (no iframes), and reach SharePoint, OneDrive, Teams and Microsoft 365 through one bearer token and one SDK.

One model · three host surfaces · zero server code in your solution

What it replaces

- Script Editor and Content Editor web parts
- JSLink field renderers and Display Templates
- SharePoint-hosted and provider-hosted add-ins
- Custom .aspx application pages with farm solutions
- No-code sandbox solutions in the old style

What it lets you build

- Client-side web parts on modern pages
- Application customizers (header / footer)
- Field customizers and list view command sets
- Adaptive Card Extensions for Viva Connections
- Teams tabs and personal apps

Hosting, in one line

Your component's JavaScript is uploaded to the tenant App Catalog, served from the SharePoint CDN, and rendered inside the live page DOM.

What you bring

Node.js + npm + TypeScript,
a JavaScript framework of your choice,
a tenant App Catalog and a developer who
already shipped code with REST or Graph.

What you no longer need

- SharePoint servers to deploy to
- Full-trust code on the server
- An on-prem SharePoint farm
- iFrames to keep custom code isolated
- A different code base per host surface

```
// same component, many hosts.
```

Evolution: server → add-in → client-side

Twenty years of customisation, three big shifts in trust model. Today it is end-user, current-user, in-browser.

pre 2013

Server-side

Farm solutions, .NET assemblies, full-trust code deployed to every front-end server. Powerful, heavy, slow to upgrade, and unsafe outside SharePoint's own boundaries.

2013 — 2024

Add-in model

Provider-hosted or SharePoint-hosted apps, run elsewhere or in an iframe. Solved the trust problem. Added OAuth dance and a user-visible permission prompt every time.

2016 — today

SPFx · client-side

Auto SSO with the signed-in user. In the live page DOM, no iframe. React by default, framework-agnostic. App Catalog hosts the bundle, CDN serves it, Graph does the talking.

2025 →

Heft + Graph-first

SPFx 1.22 swaps gulp for Heft. New rigs, TypeScript 5.8, cleaner *npm audit*. Microsoft Graph is the primary data layer. SPA-quality toolchain, lowest trust cost so far.

What changed in the trust model

Where the code runs

- Farm · in IIS on every WFE
- Add-in · provider-hosted in Azure or in an iframe
- SPFx · in the page DOM, current user session

Who signs the call

- Farm · the app pool identity
- Add-in · the add-in principal with OAuth
- SPFx · the signed-in user, automatically

Status today

- Farm · not supported in SPO
- Add-in · retired, no new investment
- SPFx · the recommended path

How SPFx fits together

Five stages from your laptop to a web part rendered live inside a Microsoft 365 page.

From source to live component

1 · Source

TypeScript + React (or your framework), Sass, JSON manifests, assets.

```
src/
config/
.yo-rc.json
package.json
```

2 · Build

Heft or gulp phases: lint, type-check, bundle, manifest, package solution.

```
heft build
heft package-solution
# ship builds the sppkg
```

3 · App catalog

Tenant admin uploads the .sppkg to the App Catalog and approves API permissions.

```
/sites/appcatalog
Deploy & approve
Tenant-wide or site-scoped
```

4 · Serve

JS bundle hosted on the tenant App Catalog CDN. Browser downloads, runs, signs every call.

```
publiccdn.sharepoint.com
tenant-private CDN
auto-loader.js
```

5 · Render

Live page DOM, bearer-token calls to Graph and SharePoint, on-property-change re-renders.

```
onInit()
render()
onPropertyChange()
```

Where the trust comes from

Microsoft Entra token

Issued for the signed-in user. No app-only prompts.

SpHttpClient + MSGraphClientV3 read it for you.

Tenant App Catalog approval

Admins gate consent and decide scope.

Tenant-wide, site collection, single site.

Heads-up

There is no server-side runtime in your codebase. Approval, hosting and permissions happen in the tenant; code lives in the App Catalog; data lives in Microsoft 365.

Workbench vs hosted page

Two hosts, one component. The local workbench iterates fast; the hosted page proves reality.

Local workbench · iterative dev

A playground page served from `localhost:4321`. It hosts your web part off the dev bundle and reloads on save. You see the same renderer ship in production.

```
$ heft start --serve
> Serving at https://localhost:4321
# open /_layouts/15/workbench.aspx
```

Hosted page · real test

A modern page on the real tenant. The component loads from the App Catalog CDN, uses real tokens and real data. Preview or deployed to a test site first.

```
https://contoso.sharepoint.com/sites/hr
  /SitePages/Testing.aspx
> web part loads from real CDN
```

The 4 lifecycle hooks you'll write

Web parts implement a small base class. Four hooks cover the interesting moments; everything else is React.

onInit()
Async bootstrap
Configure providers, pre-fetch config.

render()
Build a React element
Returns JSX. Cheap to call.

onPropertyChange()
Reacts to edits
Author edits the property pane.

onDispose()
Tear down cleanly
Unsubscribe, log, recycle.

```
// fine-grained hooks exist too: load,
serialize, deserialize,
getPropertyPaneConfiguration.
```

Node, TypeScript, version matrix

A code-compatibility table for the four SPFx versions you will meet today, plus Node 22 LTS as the single supported line.

Compatibility at a glance

Numbers from Microsoft release notes. Pin Node 22 LTS — even-numbered active LTS lines are the only supported track.

SPFx	Node	TypeScript	Toolchain	Released
1.20.x	18 LTS	5.x (4.7.4 default)	gulp	Mar 2025
1.21.x	22 LTS ↑ only	5.x	gulp (legacy)	Jul 2025 · our baseline
1.22.x	22 LTS	5.8 default	Heft + rig	Dec 2025
1.24.x	22 LTS	5.x	Heft only	2026 (planned)

React 17 is the default in 1.21; React 18 from 1.24+; React 19 supported but optional. Always pin Node in `.nvmrc` — nvm is non-negotiable.

Three rules we pin

01

Stay on Node 22 LTS

Anything 18 or 24 not even-LTS breaks the Heft rig and several MS packages.

02

Use nvm, not n

Windows and macOS agree on nvm. Per-folder .nvmrc keeps teammates aligned.

03

Don't pin TypeScript yourself

Let the generator's devDependencies pick it. Override only on reviewed need.

Where we land today

The day's lab uses SPFx 1.21 with gulp for clarity, then we sketch the v1.22 Heft path on slide 17.

Toolchain story — gulp, Heft, Vite

Old, current and emerging build paths: the one your team picks is less important than understanding what each fixes.

G gulp · 1.0 — 1.21

Stability and familiarity

- gulpfile.js + custom Tasks
- Webpack under the hood
- Critical fixes only after 1.22
- `gulp serve` / `gulp bundle`
- `gulp package-solution`

Sample tasks

```
$ gulp trust-dev-cert
$ gulp serve --nobrowser
$ gulp bundle --ship
$ gulp package-solution --ship
```

H Heft · 1.22 →

Config-driven, plugin-based

- JSON config in `./config`
- Inherits the SPFx rig package
- Plugin model for lint, jest, sass
- Phases + tasks replace gulp pipeline
- Default for new projects from 1.22

Sample commands

```
$ heft start
$ heft build
$ heft package-solution
$ heft test
```

V Vite-era preview

Hot reload, faster dev loop

- `spfx-fast-serve` proxies dev bundle
- 10–15× faster rebuild cycles
- Hot Module Reload via Vite dev
- Compatible with gulp or Heft
- Optional, not the default

Tip

```
$ npm i -D fast-serve
$ fast-serve
# adds a vite-based dev server
# production build still uses gulp/heft
```

Yeoman generator flow

One command, six prompts, a complete project. Pick the answers with intent.

Install & invoke

```
$ npm install -g yo gulp-cli
$ npm install -g @microsoft/generator-sharepoint
$ mkdir hrlite && cd hrlite
$ yo @microsoft/sharepoint
```

Solution name

`hrlite-onboarding`

Becomes the sppkg file name and folder. Pick once.

Target

`SharePoint Online only`

On-prem 2016 / 2019 still supported, separate path. Pick the closest match.

Component type

`WebPart · Extension · Library`

Library is a reusable component with no UI, ship it under webparts.

Web part name

`OnboardingTracker`

Camel case, no spaces. Goes into the manifest id and the class name.

Framework

`React · No JavaScript framework`

React is the default. Pick "no framework" when you author HTML yourself.

Test surface

`Local · Online · both`

Decide whether you trust dev cert early; rest can be added later.

What you receive

A working TypeScript project

with one web part, a config folder, an .yo-rc.json and a README that runs.

Re-run later

```
$ yo @microsoft/sharepoint
    --solutionName hrlite
    --componentType webpart
    --framework react
```

Note on flags

`--plusbeta` for Microsoft 365 Copilot App scaffolding. Run from a project root, not your home directory.

Anatomy of a generated project

Every folder earns its place. Memorise this tree; everything in v1.21 maps onto v1.22 with renamed top-level scripts.

Annotated file tree

```
hrlite · SPFx 1.21 / 1.22
config/     · serve.json, package-solution.json
sharepoint/assets/ · icons and bundles that ship to tenant
src/
  index.ts · registers web parts with the manifest
  webparts/<name>/
    <Name>WebPart.ts · the class derived from BaseClientSideWebPart
    <Name>WebPart.manifest.json · id, title, properties, previews
    components/ · one folder per React component
    loc/ · one .d.ts per language for localised strings
  services/ · data access without poking web parts directly
.yo-rc.json · answers kept for future re-runs
.nvmrc · pins Node 22 for teammates & CI
.vscode/ · launch + tasks for F5 debugging
package.json · dependencies, scripts, version
tsconfig.json · extends the rig; rarely edited by hand
README.md · scaffolded but useful
```

Manifest is the contract

The .manifest.json of every component carries its id, version, properties and previews. Idempotent & tenant-stable across upgrades.

Components live under webparts

One folder per visually-rendered React component. Web part owns the React tree, not the page.

services/ keeps data calls out of UI

One class per data source. PnPjs setup, error mapping, mock-friendly — testable from Jest.

.nvmrc is not optional

Pin 22 in every repo. CI installs with the same version or builds silently fail.

React with SPFx

Framework-agnostic on paper, React in practice. Function components + hooks do 90% of the work and stay readable.

Why React ships by default

Component-shaped data

Microsoft 365 surfaces are lists, cards, dashboards. React's tree model matches page composition; teams already know it from Power Apps.

Hooks cover the lifecycle

`useState`, `useEffect`, `useMemo` cover state, side effects and derivation. Class web parts are still legal but rarely needed.

Build a UI library once

Fluent UI React (`@fluentui/react`) and PnP controls slot straight in, look right, behave right.

Same skillset, more hosts

Code you write today dies in Teams, Outlook, Office.com, Viva with near-zero porting.

A first web part, in 30 lines

OnboardingTrackerWebPart.tsx

```tsx
import * as React from 'react';
import { PrimaryButton } from '@fluentui/react';
import { spfi, SPFx } from '@pnp/sp';
import { WebPartContext } from '@microsoft/sp-webpart-base';

type Hire = { Id: number; Title: string; Stage: string };

const Board: React.FC<{ ctx: WebPartContext }> = ({ ctx }) => {
  const [hires, setHires] = React.useState<Hire[]>([]);

  React.useEffect(() => {
    const sp = spfi().using(SPFx(ctx));
    sp.web.lists.getByTitle('Hires')
      .items.select('Id,Title,Stage')()
      .then(setHires);
  }, [ctx]);

  return (
    <ul>{hires.map(h => (
      <li key={h.Id}>{h.Title} — {h.Stage}</li>
    ))}</ul>
  );
};
```

Hooks, props, state, render

The render flow as a four-step loop. Each property change repeats it; React memo handles 95% of the cost.

What happens between “props change” and “DOM commits”

1 · Props arrive

Web part passes new typed props on every property change. Function component receives them as the first argument.

2 · State + effects

useEffect schedules a fetch when ctx is undefined. Cancellation flag prevents “set on unmounted” warnings.

3 · Reconcile

React computes a virtual diff. Pure components and stable keys keep this cheap; lists need stable Id fields.

4 · Commit

DOM updates inside React’s portal in the page DOM. SPFx tracks the root for clean-up.

Hooks you will reach for

useState — local pieces of state, one call per concern.
useEffect — fetch on mount, with cleanup. Empty deps for “once”.
useMemo — cache derived values when computation is heavy.
useCallback — stabilise functions passed as props.
useRef — imperative DOM, focus, scroll position.

Class web parts are still legal

- Legacy projects still extend `BaseClientSideWebPart`.
- For new components, return JSX from `render()`.
- Move side-effects out of the web part into a React component.
- Test components with React Testing Library; web parts with jest only when you must.
- If unsure, build the React component first, then wrap.

Web parts: manifest + lifecycle

A web part is one class, two contracts: a manifest shipped to the App Catalog and a hosted lifecycle method set.

Manifest anatomy

```
{
  "$schema": "https://developer.microsoft.com/json-schemas/spfx/client-side-web-part-manifest.schema.json",
  "id": "f0e3a5b4-...",
  "version": "1.0.0.0",
  "alias": "OnboardingTrackerWebPart",
  "componentType": "WebPart",
  "manifestVersion": 2,
  "preconfiguredEntries": [{
    "groupId": "5c03119e-3074-46fd-...",
    "group": { "default": "HR" },
    "title": { "default": "Onboarding Tracker" },
    "description": { "default": "New hires, where they are" },
    "officeFabricIconFontName": "People",
    "properties": {
      "listTitle": "Hires",
      "viewName": "All Items"
    }
  }]
}
```

Lifecycle, in order

`onInit()`

construct, prepare providers

`render()`

return a React element

`onPropertyChange()`

author re-edits the property pane

`getPropertyPaneConfiguration()`

declares editable properties

`onDispose()`

unset event listeners

Also worth knowing

- `loadPropertyBag`
- serialize / deserialize
- onAfterDeserialize

Property pane patterns

The pane is just a configuration form. Choose between built-in controls, custom controls and PnP, then decide live or on-apply.

Three layers of control

1 · Built-in basic controls

Text, dropdown, toggle, slider, multi-select, choice, checkbox. Cover 80% of needs; ship the day you design the form.

2 · Custom · built with React

You implement `IPropertyPaneField`. Draw whatever you can picture; has access to web part context.

3 · PnP @pnp/spfx-property-controls

Pre-built rich controls: list picker, people picker, date / time, file picker, term picker, sortable list. Drop them in.

Reactive vs non-reactive

Reactive

Apply the change as you type or toggle. Web part re-renders live. Default in most controls.

Non-reactive

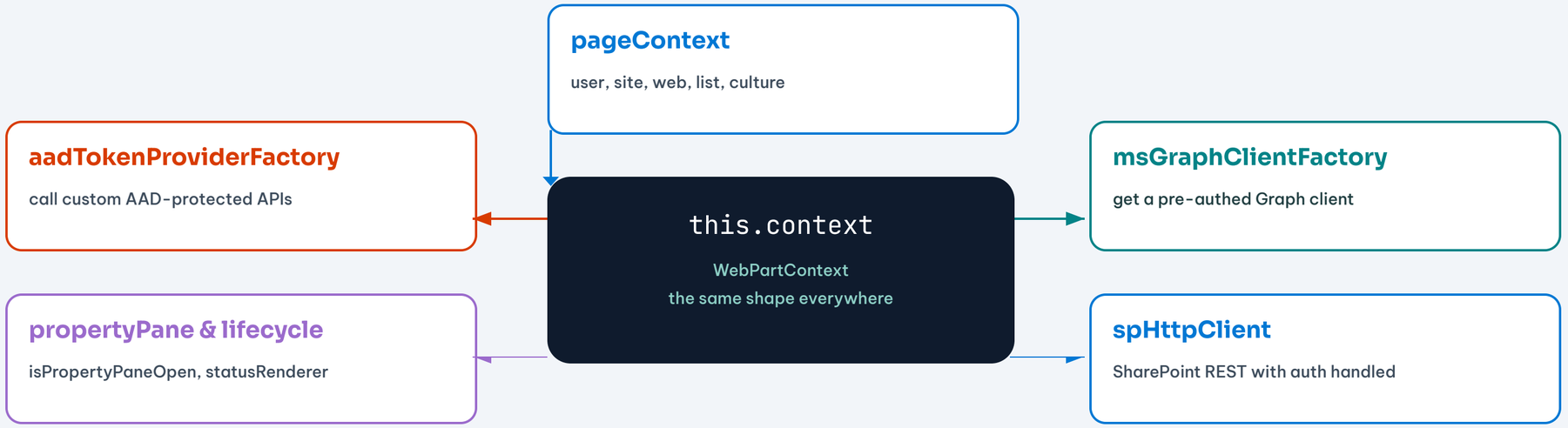
Only commit when the user clicks Apply. Useful for slow-loading dependencies.

Wiring a property to web part state

```
protected get propertyPaneSettings(): IPropertyPaneSettings {
  return {
    pages: [{
      header: { description: 'Configure the dashboard' },
      groups: [{
        groupFields: [
          PropertyPaneTextField('title', { label: 'Title' }),
          PropertyPaneToggle('showCount', { label: 'Show count' })
        ]
      }]
    }]
  };
```

The `this.context` object

One object — five surfaces. Whatever the page knows about its user, site and tenant, this object surfaces.



A useful default pattern

```
const { pageContext, spHttpClient, msGraphClientFactory } = this.context;
const me = pageContext.user; // { displayName, loginName, email, ... }
const siteId = pageContext.site.id.toString();
const graph = await msGraphClientFactory.getClient();
const token = await aadTokenProviderFactory.getTokenProvider().getToken('https://graph.microsoft.com');
```

Three paths to data

SPHttpClient + REST, MSGraphClientV3 + Graph, PnPjs. Pick by reach, not by hype.

R SharePoint REST

via SPHttpClient

- Tightest fit for SP-only nuance
- Returns *SP.Data.** types
- Verbose, low-level, easy to misuse
- Lives at `/_api/`
- Best for one-offs and migrations

G Microsoft Graph

via MSGraphClientV3

- Cross-M365 (mail, Teams, identity)
- Standard `$expand`, `$filter`
- Consistent types across surfaces
- Watch throttling; use delta for sync
- Best when M365 is the answer

P PnPjs

fluent wrapper, TS types

- Covers REST + Graph + Teams
- Fluent, batched, type-safe
- Best DX of the three
- Active PnP community
- Default for new components

Same call, three styles

```
// A. SPHttpClient + REST
const r = await spHttpClient.get(
  `${pageContext.web.absoluteUrl}/_api/web/lists/...`, SPHttpClient.configurations.v1);

// B. MSGraphClientV3 + Graph
const g = await graph.api('/me').get();
```

Default we recommend

- Start with PnPjs for everyday SharePoint / Graph calls.
- Drop to MSGraphClientV3 when PnPjs doesn't model the surface.
- Drop to SPHttpClient only for SP-specific endpoints PnPjs doesn't cover.
- Wrap them all in a service — never in a component.

Loading data with React hooks

Three state slots, a single effect, error and cancel-aware. The pattern repeats across every list, every fetch.

Loading a list with state, refs and effect

```tsx
const ListView: React.FC<{ ctx: WebPartContext; list: string }> = ({ ctx, list }) => {
  const [items, setItems] = React.useState<Item[]>([]);
  const [error, setError] = React.useState<string | null>(null);
  const [busy, setBusy] = React.useState(true);

  React.useEffect(() => {
    const sp = spfi().using(SPFx(ctx));
    let cancelled = false;

    sp.web.lists.getByTitle(list).items
      .select('Id,Title,Status,Modified')
      .orderBy('Modified', false)
      .top(50)()
      .then(rows => { if (!cancelled) setItems(rows); })
      .catch(e => { if (!cancelled) setError(e.message); })
      .finally(() => { if (!cancelled) setBusy(false); });

    return () => { cancelled = true; };
  }, [ctx, list]);
```

Four things to keep

1 · Three states

data, error, busy — one per concern. Never collapse them.

2 · Cancel on unmount

Flag in closure; check it before setState. Avoids the React warning storm.

3 · Memo the deps

Props that trigger effect: `ctx`, `list`. Add or remove carefully.

4 · Wrap the renderer

busy and error drive the UI — skeletons, retry buttons, empty state.

```
// Keep the web part dumb; let the component own state.
```

Lab: from scaffold to deploy

Four phases, one hrlite-onboarding web part, end-to-end on real data. Estimated time: 60–90 minutes.

Linear journey with one feedback loop

phase A

Scaffold & iterate

Yeoman, React template, open in VS Code. Trust the dev cert, run *gulp serve* on the workbench.

phase B

Code the widget

One functional component, hooks for state, PnPjs for a list. Add a property pane. Add error states.

phase C

Bundle + package

Build with *--ship*, package the .sppkg, deploy via CLI for Microsoft 365 or PnP PowerShell.

Phase D · ship and verify on a real page

Upload the .sppkg to the tenant App Catalog. Add the web part to a modern page. Verify with a real user account, not a guest.

Add to App Catalog

```
sites/appcatalog
Deploy + check API access
```

Add to a page

```
Edit / HR dashboard
Pick "Onboarding Tracker"
```

Watch the network

DevTools, Network tab. Graph and REST tab.

Bump version and repeat

Loop back to Phase A — shipping is the milestone.

Lab A — scaffold and run

Phases A and B in one screen: eight real CLI commands you'll type, in order.

Commands, in order

```
# --- one-time global install ---
$ nvm install 22 && nvm use 22
$ npm install -g yo gulp-cli
$ npm install -g @microsoft/generator-sharepoint

# --- scaffold ---
$ mkdir hrlite && cd hrlite
$ yo @microsoft/sharepoint
# solution: hrlite-onboarding
# target: SharePoint Online only — WebPart — React

# --- run locally ---
$ npm install @pnp/sp @pnp/graph @fluentui/react
$ gulp trust-dev-cert
$ gulp serve --nobrowser
> open https://<tenant>.sharepoint.com/_layouts/15/workbench.aspx
```

Flags worth remembering

`--nobrowser` lets you stay in the terminal.
`--reset` clears the spkg before rebuild.
`--use-gulp` — opt out of Heft in 1.22.

Trust the dev cert

One cert per machine. Run once per OS user account. Don't ship the cert to colleagues; let them trust their own.

Done when

- Workbench shows your web part
- *node --version* prints 22.x
- Browser console shows your title

Lab B — build, deploy, verify

Production bundle, package-solution, App Catalog upload, then add to a page. Plus a v1.22 Heft ribbon.

Bundle, package, deploy

```
# 1. Build a production bundle
$ gulp bundle --ship
  # emits sharepoint/solution/hrlite-onboarding.sppkg

# 2. (optional but recommended) wrap with the PnP tool
$ m365 spfx package generate --output hrlite.spkg

# 3. Upload to the tenant App Catalog
$ m365 spo app add --filePath hrlite.spkg

# 4. Approve API access in the App Catalog if needed
$ m365 spo app deploy --name hrlite-onboarding

# 5. Add the web part to a page, edit it live
```

Expected after step 5

- Web part appears under "HR" group, "Onboarding Tracker" title
- Adding it to /sites/hr-lite pulls latest items
- Property pane reactively filters by list and view

v1.22 Heft ribbon

```
$ heft start
$ heft build
$ heft package-solution
$ heft test --watch
```

Same artefacts, config-driven. -p @pnp/cli-microsoft365 speeds up upgrades.

Verify before you claim done

- Sign-in with a non-guest account
- Permission looks right on the page
- Hard refresh — not the workbench
- DevTools shows your class name
- Edit page, change property live

Three industry scenarios

How today's SPFx patterns ship inside real projects — retail store-ops, banking approvals and HR onboarding.

Retail · Inventory dashboard

Northwind Stores, 320 shops

A regional manager opens her store hub and sees today's low-stock count, the top five movers, and reorder links — one React web part per region.

Stack: PnPjs + Graph /sites + day-1 PnP provisioning for the per-store site tree.

Approach

1. read inventory rows scoped with region filter
2. cache last-hour snapshot via PnP caching
3. render three KPI tiles + reorder CTA
4. property pane picks region from hub list

Outcome: 320 stores, zero add-ins, 1 hub per region.

Bank · Document approval

Provident Bank, regulated

An approver opens a drawer page, sees top-of-stack with severity and SLA timers, and approves or rejects inline. Audit entries write back through Graph.

Stack: PnPjs + Graph approvals + an SPFx application customizer for the drawer header.

Approach

1. resolve approvals via Graph /identityGovernance
2. render list sorted by tenant due date
3. approve or reject — PATCH with optimistic UI
4. subscribe via delta for live state

Outcome: 6 minutes saved per approval, audit-grade.

HR · Onboarding tracker

Acme Corp, 1 800 hires / yr

HR & hiring managers open a per-cohort page and see where every new hire is. Tasks mark themselves done and write back to the source list.

Stack: PnPjs /lists/items + Graph /me + a parent PnP provisioning pipeline (Day 1).

Approach

1. scaffold once with the lab web part
2. group rows by Stage (PRIDE / OPS / DONE)
3. click a card → PATCH in place
4. pin this lab's component, reuse forever

Outcome: 14-minute onboarding, one tenant App Catalog.

Day 2 recap & looking ahead

Five ideas you should remember from today — and four topics that arrive next.

Five takeaways

1. **SPFx is just JavaScript in the page**
   No servers to deploy, no iframes, current user, bearer token, in-DOM.
2. **React is the default; function components + hooks do the work**
   Class web parts still work; new components return JSX and use useState/useEffect.
3. **Pick data by reach, not by novelty**
   PnPjs in daily use, Graph when M365 is the surface, SPHttpClient for edge cases.
4. **Toolchain moved — track it calmly**
   Pin Node 22. Heft is now default for new projects; gulp remains supported.
5. **Day 1 + Day 2 = one practice**
   PnP provisioned the canvas; SPFx paints it; Power Platform and search still ahead.

Day 3 in four blocks

From render to operations.

Block 1 · Power Platform in SharePoint
Power Apps custom forms on lists, Power Automate approvals and notifications, custom connectors that wrap SPFx endpoints.

Block 2 · Search and content intelligence
SharePoint search schema, query variables, vertical results, Graph connectors and external content ingestion.

Block 3 · ALM, governance and operations
Git, dev / test / prod promotion, tenant governance checklist, versioned spkgs, telemetry and Feature Pack notes.

Block 4 · Viva, Teams, Copilot
Adaptive Card Extensions for Viva, Teams tabs, SharePoint Copilot Apps preview, future-of-the-platform demo.